Zeynep Akçil

22102833

Section 2 | 14.05.2023

Lab 6: Greatest Common Divider

Purpose

The purpose of this lab is to design a circuit that calculates the greatest common divisor (GCD) of two 8-bit numbers. In the meanwhile, registers and finite state machines will be studied and used during the design process.

Methodology

Various different greatest common divisor finding algorithms are present. In this lab, the subtraction method will be used. It is chosen because in the previous labs, subtraction algorithm is written and implemented so it will be more practical to implement the same algorithm. Also this method is quite simple to use. It is based on subtracting the smaller number from the bigger number and continuing subtractions with the resultant and smaller numbers one after another. The steps are simply illustrated below:

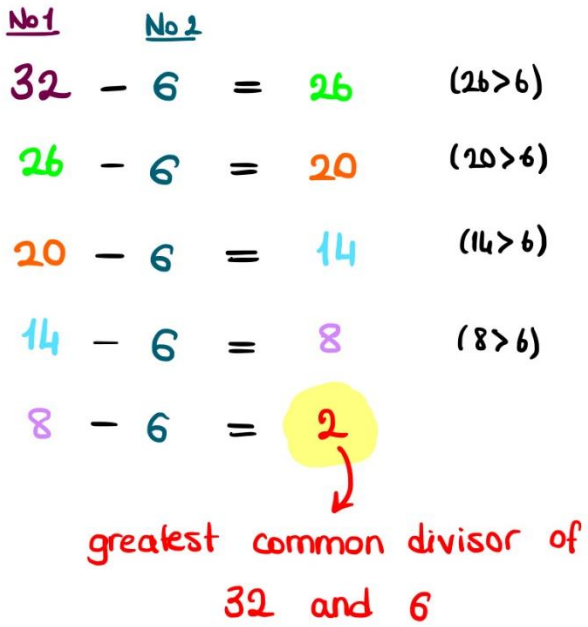

Figure 1: Subtraction method example

To clarify, steps can be ordered as followings:

1- 2 input numbers are compared.

2- If equal, GCD equals to them, algorithm is terminated.

3- If not equal, smaller one is subtracted from bigger one.

4- Turned back to Step 1, input numbers are resultant and subtracted number. As long as resultant number is greater than 0, keep repeating the steps.

5- Last resultant number greater than 0 is the greatest common divider of two inputted numbers.

In order to implement this design into FPGA code, finite state machine will be used. A Moore machine will be designed. There will be 3 states for 3 different actions in total which are maintain, shift, subtract. Also the process will be controlled by enable and reset buttons by the outer world.

Design Specifications

The design will be consist of 3 modules which are “GCD.vhd”, “comparator.vhd” and “subtractor.vhd”. Subtractor is a 8-bit ripple carry subtractor. It has 2 inputs and 2 outputs. Inputs are 8-bit binary numbers entitled as std_logic_vector and named as a and b. Outputs are result which is 8-bit binary number and check entitled as std_logic. Comparator is the second sub module. It has 3 subparts to check each case equal, greater than and less than. It also has 2 input 1 output 8-bit binary numbers named as a, b and result, respectively. By using XNOR gates between a and b’s each bit position index, equality is checked and kept in 8-bit std_logic_vector. Than greater or less cases are evaluated and their truth condition kept in variables gt and lt, separately. The situation variable keeps the cases. If first number is greater, situation is 001; if two numbers are equal, situation is 010 and finally, if first number is less, situation is 100. GCD file is the main module which includes other two modules in it. It has 5 inputs and 2 outputs. First three input are entitled as std_logic which are clk, the internal clock of BASYS 3 having 100 MHz clock frequency, reset, enables user to control the circuit and reset it by using assigned button and the enable, similar to reset in terms of control and enables the design to operate. The other 2 inputs are a and b, 8-bit binary numbers likewise in previous modules. These numbers are inputted through switches on BASYS 3. Starting from index 7, going to 0 a is assigned to W13, W14, V15, W15, W17, W16, V16 and V17 and B is assigned to R2, T1, U1, W2, R3, T2, T3 and V2. The outputs are result, 8-bit binary number, and ready. Result is the resultant output and assigned to LED’s on BASYS 3 coded as

V14, U14, U15, W18, V19, U19, E19 and U 16 for 7th index to 0th index. The states are defined in this module and in the process states are assigned according to enable, reset and other coming values.

The states of the finite state machine are named according to their operations. Maintain is active if the two inputted numbers are equal. If not, machine moves to shift state. Shift state swap numbers if second is greater than the first then it moves to subtract state.. If numbers are equal, machine turns back to maintain state. Lastly, subtract state subtracts smaller number from bigger and it moves into maintain state.

In the end, when the machine obtains the greatest common divider, it means algorithm is end. The state, after that, stays in the maintain state. Machine is waiting for a new input data to operate again in maintain state. Ready output is an indicator for this moment. When machine made its work and displaying the results, ready signal displays this.

Results

In the figures below, elaborated design schematics are given.

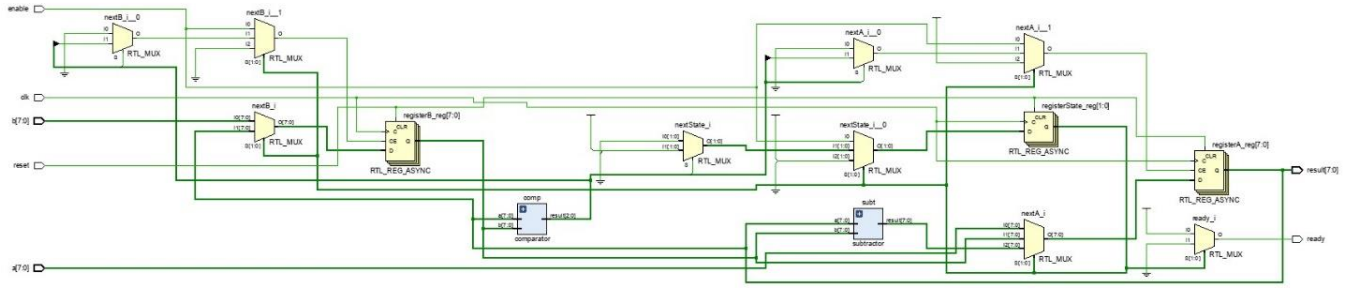

Figure 2: GCD main module schematic

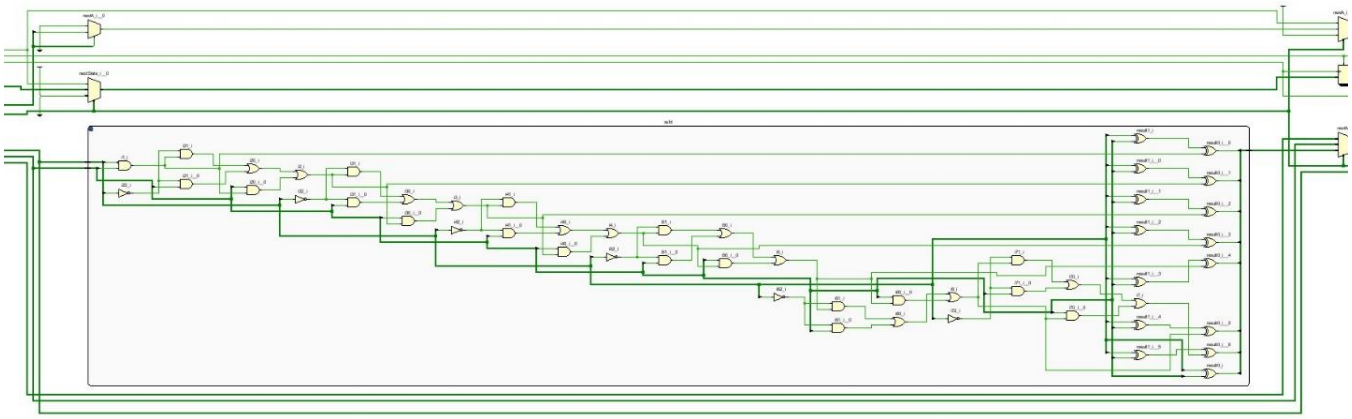

Figure 3: Subtractor module schematic

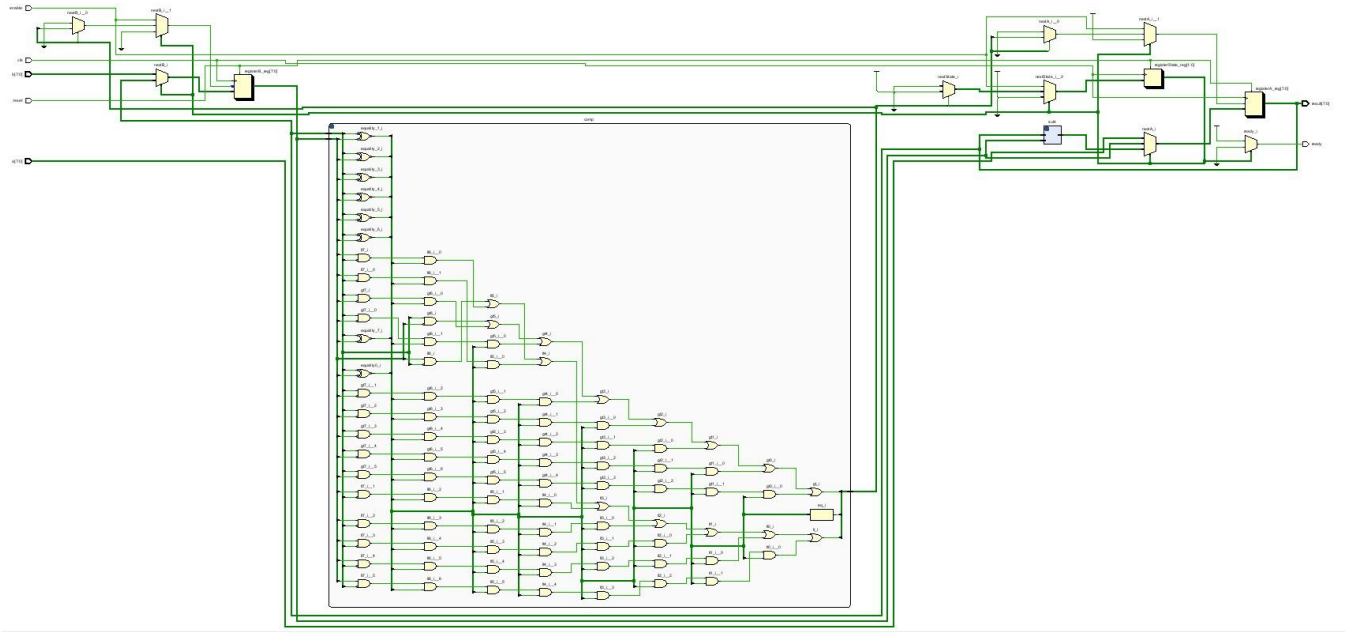

Figure 4: Comparator module schematic

Schematics are properly connected as seen above.

According to lab manual, it is requested to test the codes with input datas a=140 and b=12. In the code, a and b are 8 bit binary numbers; therefore, they need to be converted into binary as “10001100” and "00001100" for a and b, respectively. The waveform graph matches expected results.

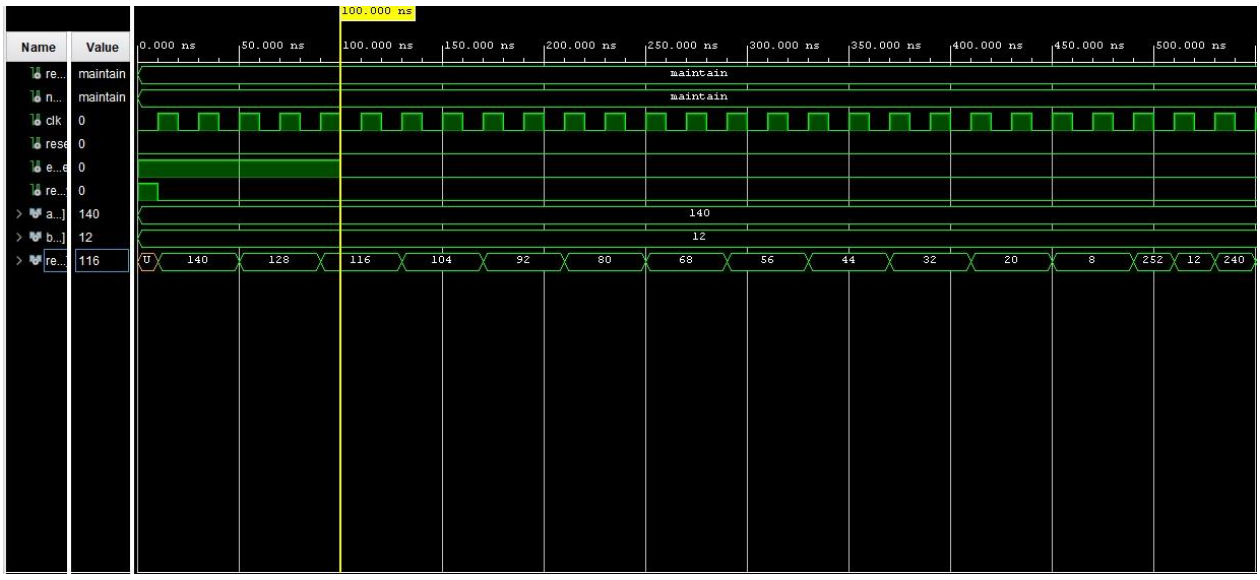

Figure 5: Waveform graph

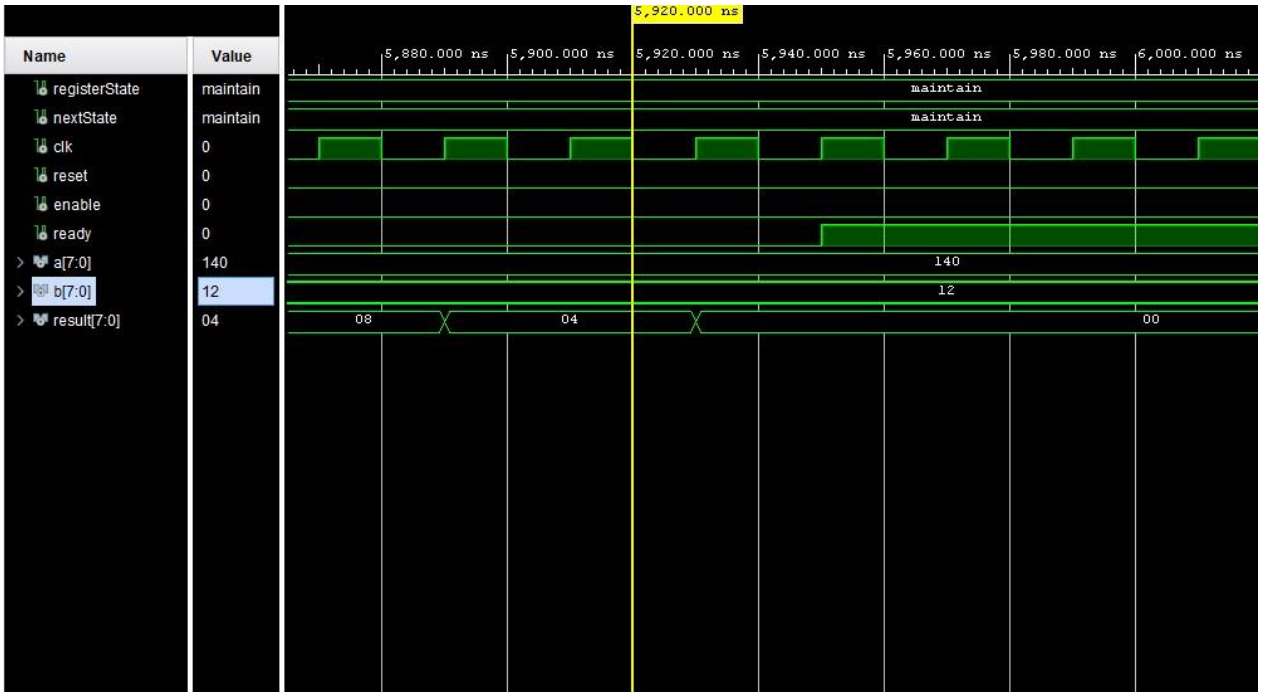

Figure 6: Waveform graph showing end of the operation

In the waveform graph seen above, the termination of the algorithm is shown. With a basic division, it can be found as 296 clock (5920/20) cycles it took to calculate GCD of 140 and 12. This might be optimized with changing state assignments. Changing state assignments may decrease the number of gates and components and therefore help to decrease the simulation part. Also the Mealy machine could help; however, since it includes bug risk, Moore machine is safer method to implement.

Conclusion

The purpose of this lab was to design a circuit that calculates the greatest common divisor (GCD) of two 8-bit numbers. In order to calculate the greatest common divisor, subtraction algorithm is used. In some sources on the Internet it is called as “Basic Euclidean Algorithm” or “Simplified Euclidean Algorithm”. However, most of the sources are explaining Euclidean Algorithm in general and it is based on repetitive mod operations; therefore, it is a different algorithm. In order to prevent the complexity, this method is called as “Subtraction Method”.

The experimental results match with the expected results.

During the process, finite state machine (FSM) is created and 3 states are used. Since there are steps after steps in the algorithm, we need a state machine that can hold data and determine next step

according to previous and current data. A combinational circuit has not that ability. And since it cannot store data, it should have been evaluate and operate by using all of the cases at the same time. In order to design such a circuit 2^18 combinations must be evaluated and included in the truth table, after that, when the expressions are obtained circuit can be designed. Obviously, it is nearly impossible to design this kind of circuit. Therefore, FSM is used. Unlike GCD module, in submodules, FSM is not needed so it is designed as combinational circuit. When combinational circuit is doable, it is wiser to choose that path because these circuits are comparatively less complex, faster and cost is less. To summarize, both ways have their benefits and drawbacks and each case should be considered separately according to inputs, outputs and expected functions.

There are two types of FSM's: Moore machine and Mealy machine. In Moore machine, operations are made in a specific step; while, in a Mealy machine, operations are made during the transition from one state to another. Mealy machine allows to implement design with less components and less clock periods; however, there might be glitches and they must be debugged and solved. Therefore, especially in complex circuits, it is wiser to use Moore machine if it is possible in terms of silicon area, cost, implementation etc.

Appendices

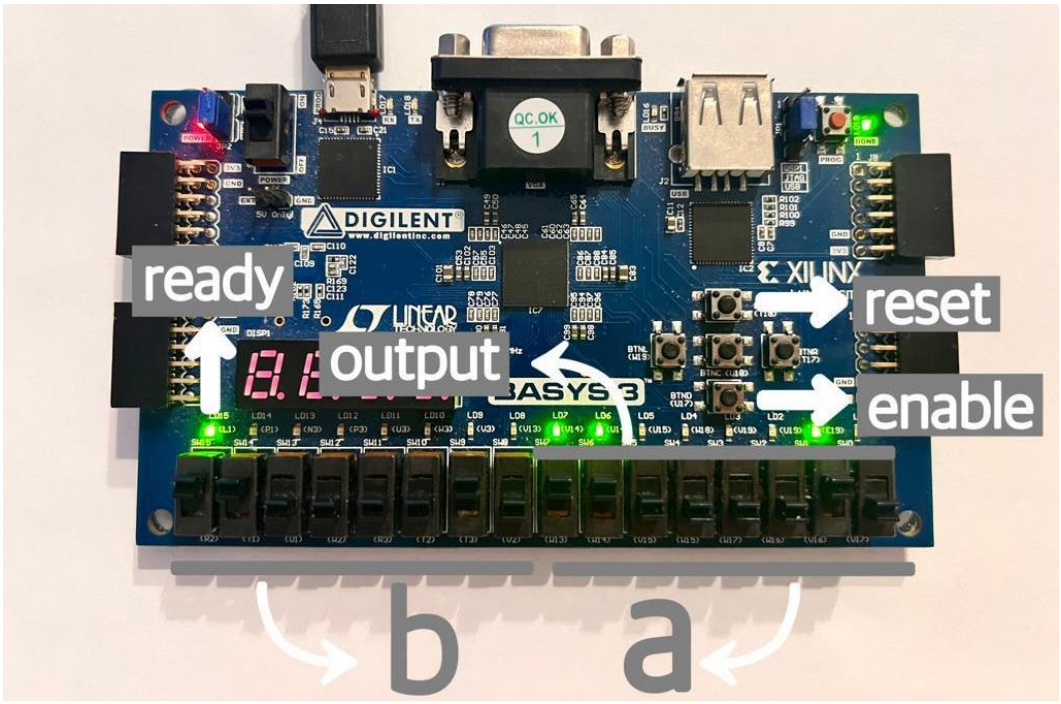

Figure 7: Input/Output details

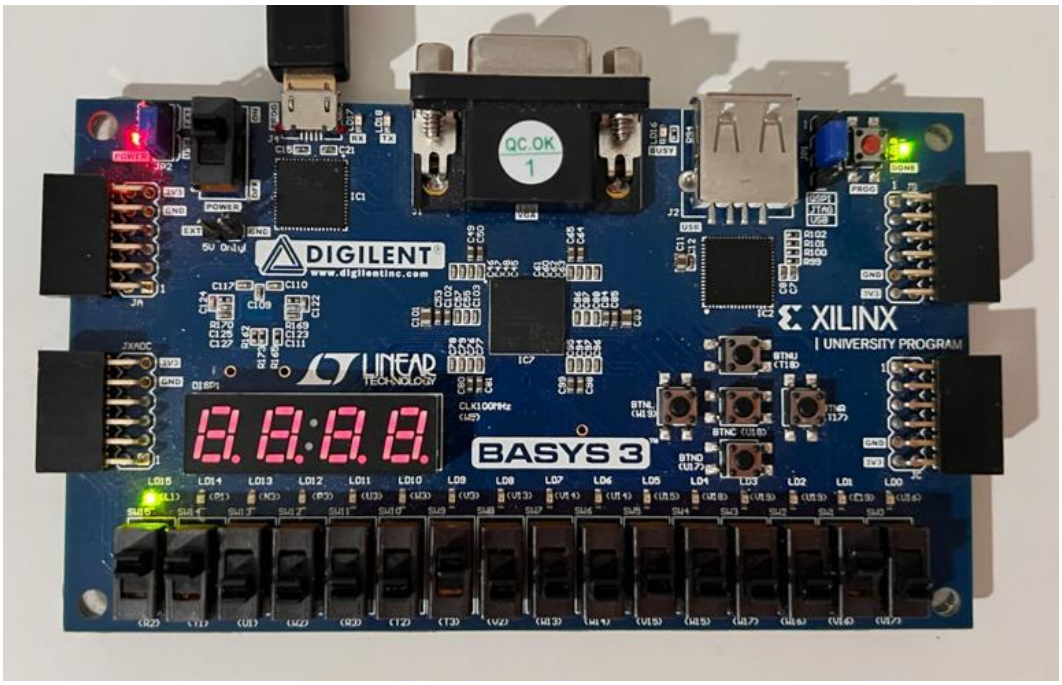

Figure 8: Output when reset button is pressed

.

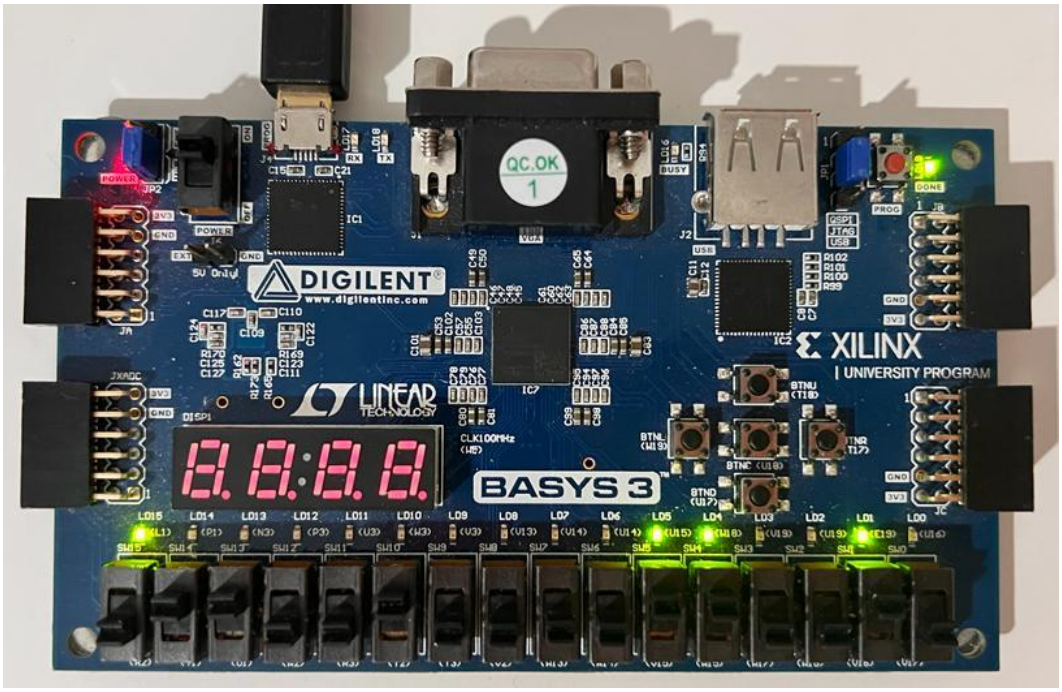

Figure 9: Example 1: a=50, b=100, GCD=50

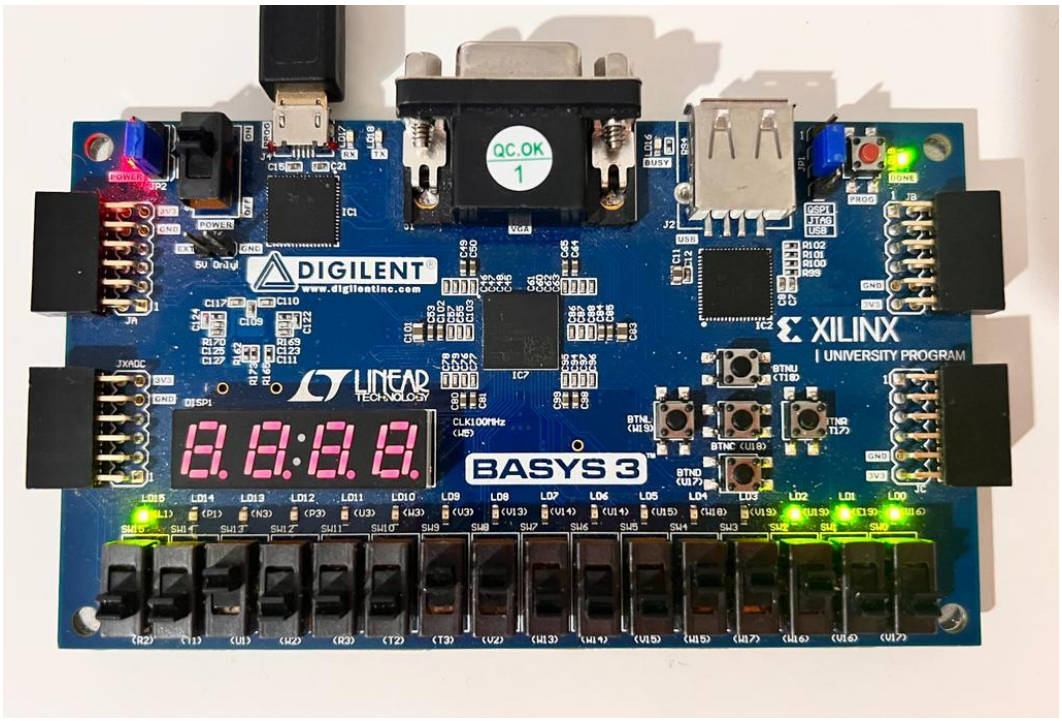

Figure 10: Example 2: a=28, b=35, GCD=7

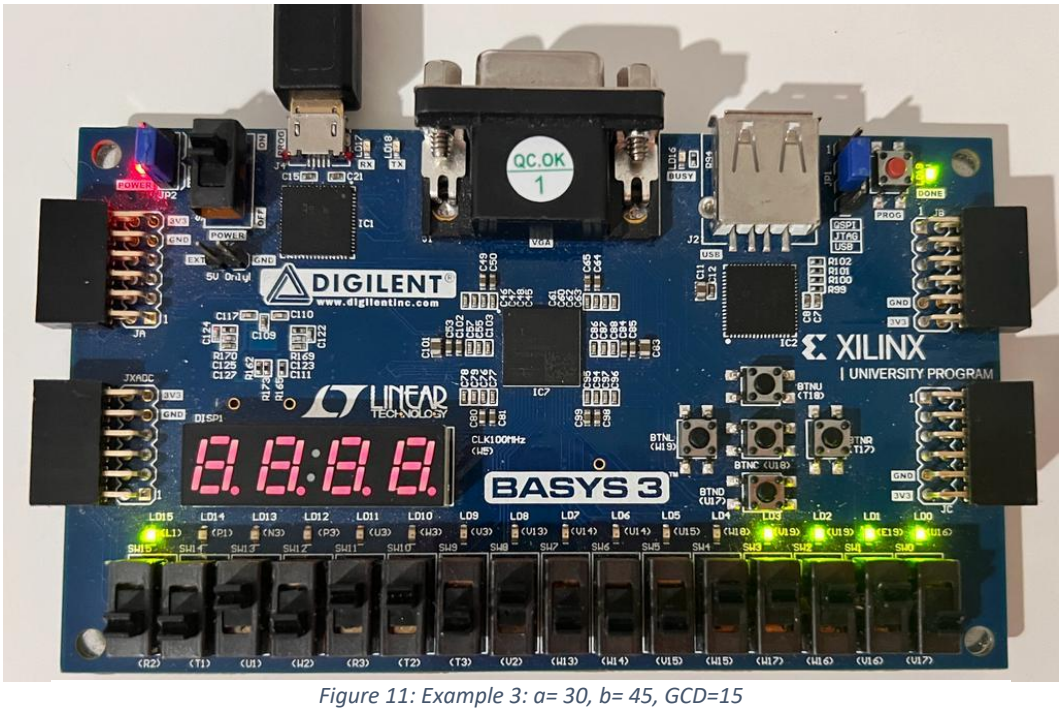

Figure 11: Example 3: a= 30, b= 45, GCD=15

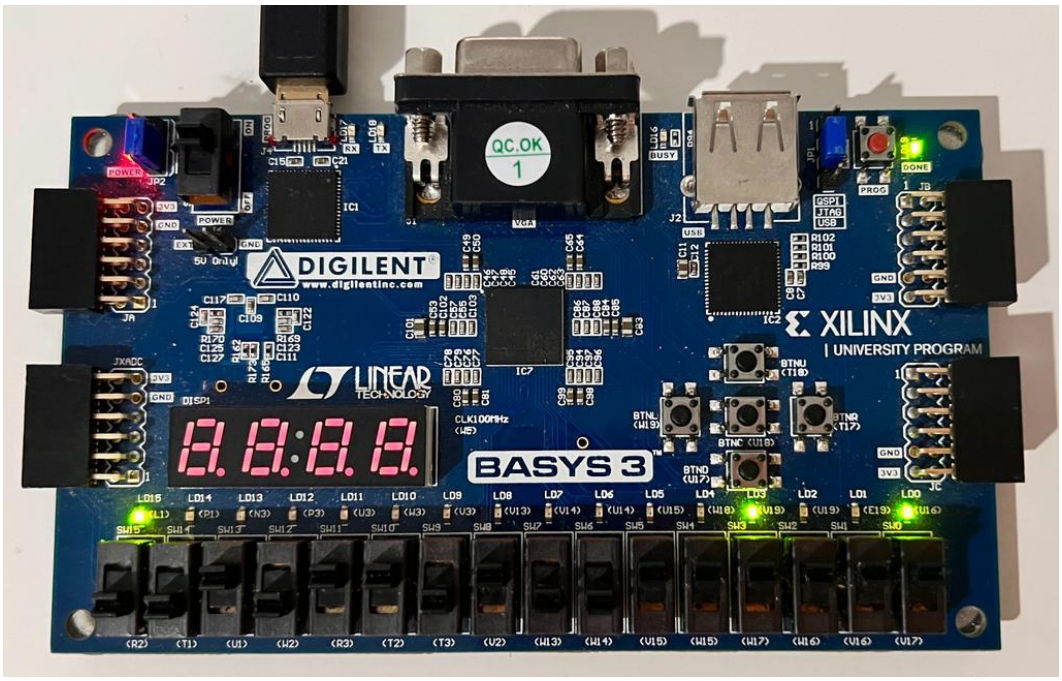

Figure 12: Example 4: a=63 , b=45 , GCD = 9

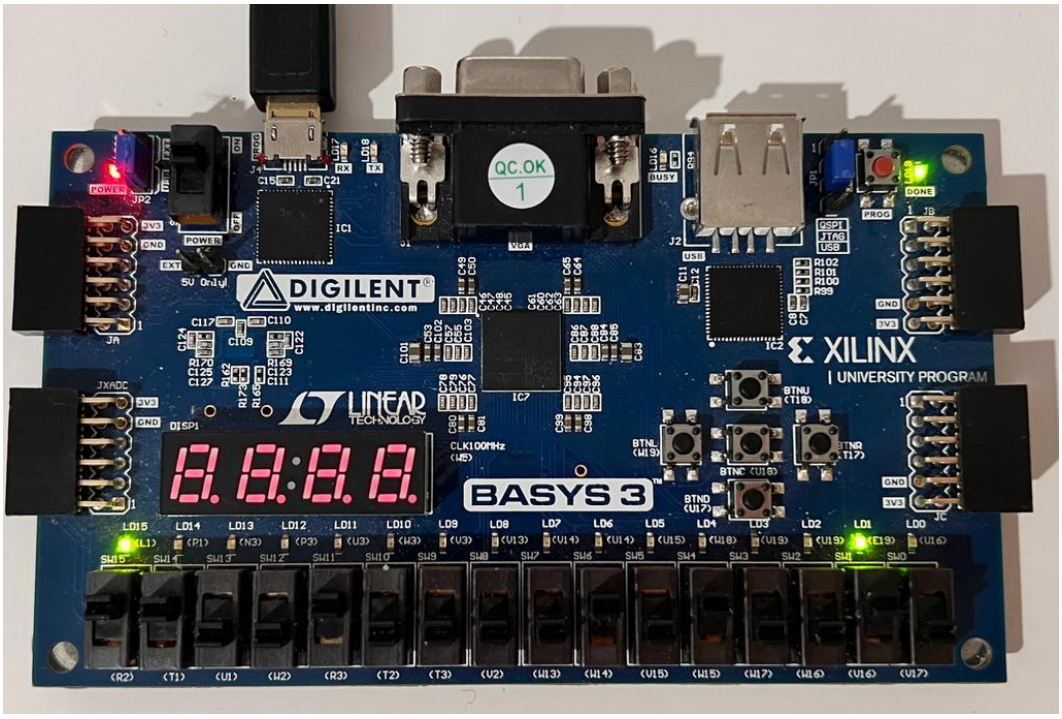

Figure 13: Example 5: a=200, b= 82, GCD =2

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use ieee.numeric_std.all;

entity GCD is
    Port (clk, reset, enable: in std_logic;
          a, b: in std_logic_vector(7 downto 0);
          result: out std_logic_vector(7 downto 0);
          ready: out std_logic);
end GCD;

architecture Behavioral of GCD is

    type state_type is (maintain, shift, substract);
    signal registerState, nextState : state_type;

    signal registerA, registerB, nextA, nextB: unsigned(7 downto 0);

    signal comparatorA, comparatorB : unsigned(7 downto 0);
    signal comparatorOut: std_logic_vector(2 downto 0);

    signal substractionA, substractionB, substractionRes: std_logic_vector(7 downto 0);

    component comparator is
        Port (a, b: in unsigned(7 downto 0);
              result: out std_logic_vector(2 downto 0));
    end component;

    component subtractor is
        Port (a, b: in std_logic_vector(7 downto 0);
```

```
        result: out std_logic_vector(7 downto 0));
    end component;
begin
    comp: comparator port map(a => comparatorA,
                              b => comparatorB,
                              result => comparatorOut);
    subt: subtractor port map(a => substractionA,
                              b => substractionB,
                              result => substractionRes);
    process1: process(clk, reset) begin
                if reset='1' then
                    registerState <= maintain;
                    registerA <= (others=>'0');
                    registerB <= (others=>'0');
                elsif rising_edge(clk) then
                    registerState <= nextState;
                    registerA <= nextA;
                    registerB <= nextB;
                end if;
            end process;
    process2: process(registerState, registerA, registerB, enable, a, b) begin
                nextA <= registerA;
                nextB <= registerB;
                comparatorA <= registerA;
                comparatorB <= registerB;
                substractionA <= std_logic_vector(registerA);
                substractionB <= std_logic_vector(registerB);

                case registerState is
                  when maintain =>
```

```
                if enable = '1' then
                    nextA <= unsigned(a);
                    nextB <= unsigned(b);
                    nextState <= shift;
                else
                    nextState <= maintain;
                end if;

            when shift =>
                if (comparatorOut(1) = '1') then
                    nextState <= maintain;
                else
                    if(comparatorOut(2) = '1') then
                        nextA <= registerB;
                        nextB <= registerA;
                    end if;
                    nextState <= substract;
                end if;

            when substract =>
                nextA <= unsigned(substractionRes);
                nextState <= shift;
        end case;
    end process;
  ready <= '1' when registerState = maintain else '0';
  result <= std_logic_vector(registerA);

end Behavioral;
```

Code 1: Main Module GCD

```
library IEEE;
```

```
use IEEE.STD_LOGIC_1164.ALL;
use ieee.numeric_std.all;

entity comparator is
   Port (a, b: in unsigned(7 downto 0);
        result: out std_logic_vector(2 downto 0));
end comparator;

architecture Behavioral of comparator is

signal eq, lt, gt: std_logic;
signal situation: std_logic_vector(2 downto 0);
signal equality: std_logic_vector(7 downto 0);

begin
    -- Checking if equal
    equality(0) <= (not a(0)) xnor (not b(0));
    equality(1) <= (not a(1)) xnor (not b(1));
    equality(2) <= (not a(2)) xnor (not b(2));
    equality(3) <= (not a(3)) xnor (not b(3));
    equality(4) <= (not a(4)) xnor (not b(4));
    equality(5) <= (not a(5)) xnor (not b(5));
    equality(6) <= (not a(6)) xnor (not b(6));
    equality(7) <= (not a(7)) xnor (not b(7));

    eq <= '1' when equality = "11111111" else '0';

    --Checking if greater
    gt <= (a(7) and not(b(7))) or
         (a(6) and not(b(6)) and equality(7)) or
```

```
        (a(5) and not(b(5)) and equality(7) and equality(6)) or

        (a(4) and not(b(4)) and equality(7) and equality(6) and equality(5)) or

        (a(3) and not(b(3)) and equality(7) and equality(6) and equality(5) and equality(4) )or

        (a(2) and not(b(2)) and equality(7) and equality(6) and equality(5) and equality(4) and equality(3))
or

        (a(1) and not(b(1)) and equality(7) and equality(6) and equality(5) and equality(4) and equality(3)
and equality(2)) or

        (a(0) and not(b(0)) and equality(7) and equality(6) and equality(5) and equality(4) and equality(3)
and equality(2) and equality(1));


    --Checking if less

     lt <= (b(7) and not(a(7))) or

          (b(6) and not(a(6)) and equality(7)) or

          (b(5) and not(a(5)) and equality(7) and equality(6) )or

          (b(4) and not(a(4)) and equality(7) and equality(6) and equality(5)) or

          (b(3) and not(a(3)) and equality(7) and equality(6) and equality(5) and equality(4)) or

          (b(2) and not(a(2)) and equality(7) and equality(6) and equality(5) and equality(4) and equality(3))
or

          (b(1) and not(a(1)) and equality(7) and equality(6) and equality(5) and equality(4) and equality(3)
and equality(2)) or

          (b(0) and not(a(0)) and equality(7) and equality(6) and equality(5) and equality(4) and equality(3)
and equality(2) and equality(1));


    situation(0) <= gt;

    situation(1) <= eq;

    situation(2) <= lt;

    result <= situation;

end Behavioral;
```

Code 2: Comparator Module

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
entity subtractor is
  Port (a, b: in std_logic_vector(7 downto 0);
       result: out std_logic_vector(7 downto 0));
end subtractor;

architecture Behavioral of subtractor is

signal i1, i2, i3, i4, i5, i6, i7: std_logic;

begin
   i1 <= (not(a(0)) and '0') or (not(a(0)) and b(0)) or (b(0) and '0');
   i2 <= (not(a(1)) and  i1) or (not(a(1)) and b(1)) or (b(1) and  i1);
   i3 <= (not(a(2)) and  i2) or (not(a(2)) and b(2)) or (b(2) and  i2);
   i4 <= (not(a(3)) and  i3) or (not(a(3)) and b(3)) or (b(3) and  i3);
   i5 <= (not(a(4)) and  i4) or (not(a(4)) and b(4)) or (b(4) and  i4);
   i6 <= (not(a(5)) and  i5) or (not(a(5)) and b(5)) or (b(5) and  i5);
   i7 <= (not(a(6)) and  i6) or (not(a(6)) and b(6)) or (b(6) and  i6);

   result(0) <= a(0) xor b(0) xor '0';
   result(1) <= a(1) xor b(1) xor i1;
   result(2) <= a(2) xor b(2) xor i2;
   result(3) <= a(3) xor b(3) xor i3;
   result(4) <= a(4) xor b(4) xor i4;
   result(5) <= a(5) xor b(5) xor i5;
   result(6) <= a(6) xor b(6) xor i6;
   result(7) <= a(7) xor b(7) xor i7;
end Behavioral;
```

Code 3: Subtraction Module

```
set_property PACKAGE_PIN W5 [get_ports clk]
        set_property IOSTANDARD LVCMOS33 [get_ports clk]
        create_clock -add -name sys_clk_pin -period 10.00 -waveform {0 5} [get_ports clk]


# Switches
set_property PACKAGE_PIN V17 [get_ports {a[0]}]
    set_property IOSTANDARD LVCMOS33 [get_ports {a[0]}]
set_property PACKAGE_PIN V16 [get_ports {a[1]}]
    set_property IOSTANDARD LVCMOS33 [get_ports {a[1]}]
set_property PACKAGE_PIN W16 [get_ports {a[2]}]
    set_property IOSTANDARD LVCMOS33 [get_ports {a[2]}]
set_property PACKAGE_PIN W17 [get_ports {a[3]}]
    set_property IOSTANDARD LVCMOS33 [get_ports {a[3]}]
set_property PACKAGE_PIN W15 [get_ports {a[4]}]
    set_property IOSTANDARD LVCMOS33 [get_ports {a[4]}]
set_property PACKAGE_PIN V15 [get_ports {a[5]}]
    set_property IOSTANDARD LVCMOS33 [get_ports {a[5]}]
set_property PACKAGE_PIN W14 [get_ports {a[6]}]
    set_property IOSTANDARD LVCMOS33 [get_ports {a[6]}]
set_property PACKAGE_PIN W13 [get_ports {a[7]}]
    set_property IOSTANDARD LVCMOS33 [get_ports {a[7]}]
set_property PACKAGE_PIN V2 [get_ports {b[0]}]
    set_property IOSTANDARD LVCMOS33 [get_ports {b[0]}]
set_property PACKAGE_PIN T3 [get_ports {b[1]}]
    set_property IOSTANDARD LVCMOS33 [get_ports {b[1]}]
set_property PACKAGE_PIN T2 [get_ports {b[2]}]
    set_property IOSTANDARD LVCMOS33 [get_ports {b[2]}]
set_property PACKAGE_PIN R3 [get_ports {b[3]}]
    set_property IOSTANDARD LVCMOS33 [get_ports {b[3]}]
```

```
set_property PACKAGE_PIN W2 [get_ports {b[4]}]
        set_property IOSTANDARD LVCMOS33 [get_ports {b[4]}]
set_property PACKAGE_PIN U1 [get_ports {b[5]}]
        set_property IOSTANDARD LVCMOS33 [get_ports {b[5]}]
set_property PACKAGE_PIN T1 [get_ports {b[6]}]
        set_property IOSTANDARD LVCMOS33 [get_ports {b[6]}]
set_property PACKAGE_PIN R2 [get_ports {b[7]}]
        set_property IOSTANDARD LVCMOS33 [get_ports {b[7]}]


#Buttons
set_property PACKAGE_PIN T18 [get_ports reset]
        set_property IOSTANDARD LVCMOS33 [get_ports reset]
set_property PACKAGE_PIN U17 [get_ports enable]
        set_property IOSTANDARD LVCMOS33 [get_ports enable]


# LEDs
set_property PACKAGE_PIN U16 [get_ports {result[0]}]
        set_property IOSTANDARD LVCMOS33 [get_ports {result[0]}]
set_property PACKAGE_PIN E19 [get_ports {result[1]}]
        set_property IOSTANDARD LVCMOS33 [get_ports {result[1]}]
set_property PACKAGE_PIN U19 [get_ports {result[2]}]
        set_property IOSTANDARD LVCMOS33 [get_ports {result[2]}]
set_property PACKAGE_PIN V19 [get_ports {result[3]}]
        set_property IOSTANDARD LVCMOS33 [get_ports {result[3]}]
set_property PACKAGE_PIN W18 [get_ports {result[4]}]
        set_property IOSTANDARD LVCMOS33 [get_ports {result[4]}]
set_property PACKAGE_PIN U15 [get_ports {result[5]}]
        set_property IOSTANDARD LVCMOS33 [get_ports {result[5]}]
```

```
set_property PACKAGE_PIN U14 [get_ports {result[6]}]
        set_property IOSTANDARD LVCMOS33 [get_ports {result[6]}]
set_property PACKAGE_PIN V14 [get_ports {result[7]}]
        set_property IOSTANDARD LVCMOS33 [get_ports {result[7]}]
set_property PACKAGE_PIN L1 [get_ports ready]
        set_property IOSTANDARD LVCMOS33 [get_ports ready]
```

Code 4: Constraint File

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity testBench is
--  Port ( );
end testBench;

architecture Behavioral of testBench is
   type state_type is (maintain, shift, subtract);
   signal registerState, nextState : state_type;

   signal clk, reset, enable, ready: std_logic;
   signal a, b, result : std_logic_vector (7 downto 0);

   component GCD is
      Port (clk, reset, enable: in std_logic;
           a, b: in std_logic_vector(7 downto 0);
           result: out std_logic_vector(7 downto 0);
           ready: out std_logic);
   end component;
begin

   main: GCD port map(clk => clk,
```

```
            reset => reset,
            enable => enable,
            ready => ready,
            a => a,
            b => b,
            result => result);

clkP: process begin
        clk <= '0';
        wait for 10ns;
        clk <= '1';
        wait for 10ns;
    end process;
 stimP: process begin
        enable <= '1';
        a <= "10001100";
        b <= "00001100";
        reset <= '0';
        wait for 100ns;
        enable <= '0';
        wait;
    end process;
end Behavioral;
```

Code 5: Test Bench

References

https://scienceland.info/en/algebra8/euclid-algorithm

chrome-extension://efaidnbmnnnibpcajpcglclefindmkaj/https://codility.com/media/train/10-Gcd.pdf

https://en.wikipedia.org/wiki/Euclidean_algorithm